

Detection of IoT Botnet Based on Deep Learning

Junyi Liu, Shiyue Liu, Sihua Zhang

School of Automation, Beijing Institute of Technology, Beijing 100081, P. R. China
E-mail: xyyh@bit.edu.cn

Abstract: In this paper, we propose a deep learning based approach for IoT botnet detection. We use the damped incremental statistics to extract basic traffic features of IoT devices and apply the Z-Score method to normalize the features. After that, the triangle area maps (TAM) based multivariate correlation analysis (MCA) algorithm is employed to generate dataset. Then we design a convolutional neural network (CNN) to learn the dataset and utilize the trained CNN to detect the traffic. The final experiments show that our approach can distinguish benign traffic and different kinds of attack traffic effectively and reaches the accuracy of 99.57%.

Key Words: IoT botnet, attack detection, multivariate correlation analysis, deep learning, convolutional neural network

1 Introduction

As one of the newly developing technologies, Internet of Things (IoT) has developed rapidly in recent years and has been gradually applied in the fields of energy, manufacturing, retail, health care and financial service. In recent years, the number of IoT devices in the world has reached 20 billion. BI Intelligence predicts that in 2025, the number of IoT devices will reach 55 billion and the related investment will exceed 25 trillion dollars in the world. With the rapid development of IoT, security problems are becoming increasingly serious. According to the survey report of Eclipse foundation, IoT security has been the top concern of the developers for three years in a row. On October 21, 2016, the American DNS service provider Dyn suffered from the worst DDoS attack in history. The attack was launched by the Mirai botnet and involved tens of millions of IP, most of which came from IoT devices. With the release of the Mirai source code, IoT botnet evolve faster. Various varieties based on Mirai have emerged successively, and the attack methods are more powerful and effective than before, the most famous ones are IoT Reaper and Satori.

In order to prevent further expansion and destruction, technicians usually conduct studies on detection, tracking, measurement, prediction and confrontation of IoT botnet, among which detection is the basis and key of follow-up studies and also the current research focus. Livadas et al. [1] are the earlier researchers who applied machine learning to botnet traffic detection. They used the naive Bayes, Bayesian network and J48 decision tree to distinguish between IRC traffic and non-IRC traffic, and then distinguished normal IRC traffic and botnet control traffic from IRC traffic. However, these three classifiers did not achieve the ideal performance. Although the Bayesian network has the best result, it only achieved the false positive rate (FPR) of 10%~20% and the false negative rate (FNR) of 30%~40%. Their work shows that simply apply the machine learning method to botnet detection cannot achieve good performance, and the inherent characteristics of botnet must be taken into account.

Considering the poor performance of traditional machine learning, Zhou et al. [2] applied deep learning to detect botnet based on HTTP protocol. They utilized the deep packet inspection (DPI) data provided by the network operator as raw data, and extracted 6-dimensional feature vectors in one-

hour time window, and then they input these feature vectors into a multi-layer perceptron (MLP) for training. Their method achieved the accuracy of 65.1%.

In the research of Kou et al. [3], deep learning was applied to the detection of BotCloud. They extracted 20-dimensional basic feature vectors of the network traffic from the UNB ISCX 2012 Intrusion Detection Evaluation dataset in a certain time window, and each feature vector was mapped to a 20×20 grayscale image. They used an improved LeNet-5 convolutional neural network to learn these image data and achieved the accuracy of 89.6%, the FPR of 11.2% and the FNR of 9.1%.

Botnet have several typical operational steps, namely propagation, infection, C&C communication and execution of attacks. In this paper, we detect IoT botnet in the last phase of the operation cycle, and the method is based on the following consideration: Although botnet produce new variations quickly and their behavior patterns will be more complex, a malicious botnet will certainly have an attack execution phase. When the botnet executes attack and the attack traffic is detected, the security system can disconnect the infected IoT devices from the network immediately to prevent further outbound attack traffic and stop the botnet from propagating. In this paper, we apply the damped incremental statistics to extract features of IoT device's inbound and outbound traffic, then the features are normalized by Z-Score method and mapped to images by the TAM-based MCA algorithm. We design an 8-layer CNN to learn the image data, and then classify the benign traffic and several kinds of attack traffic.

2 Extraction and Normalization of Basic Features

We use damped incremental statistics [4] to extract features. The advantage of this method is that features can be extracted at high speed on dynamic network traffic and only have the time complexity of $\mathcal{O}(1)$.

Let $D_i = \{v_1^i, v_2^i, v_3^i, \dots\}$ be an unbounded data sequence of network stream i where $v_j^i \in \mathbb{R}$ is the value of the j -th packet in stream i . The feature extractor maintains an array $A_\lambda^i = [\omega, S, SS, SR_{ij}, T_{last}]$ in memory for each stream i . $\lambda > 0$ is the decay factor, and let γ be the decay function:

$$\gamma = 2^{-\lambda t} \quad (1)$$

where t is the time interval between two observations. ω is

current weight and represents the number of recently captured packages. S and SS are the sum and squared sum of the value of packets captured so far. SR_{ij} is the sum of residual products between streams i and j , and defined as:

$$SR_{ij} = \sum r_i r_j \quad (2)$$

where

$$r_i = v_{cur}^i - \mu_{D_i} \quad (3)$$

and μ_{D_i} is defined as formula (4). r_j has the same form with r_i . T_{last} is the timestamp of the last update of A_λ^i .

Each element of A_λ^i has an initial value of 0, and the method to update A_λ^i with v_{cur}^i the current value of stream i and r_j the residual of stream j at time T_{cur} is shown as follow:

Algorithm: Update A_λ^i

Input: A_λ^i before update, T_{cur} , v_{cur}^i , r_j

Output: Updated A_λ^i

1) $\gamma \leftarrow 2^{-\lambda(T_{cur}-T_{last})}$

2) $A_\lambda^i \leftarrow [\gamma\omega, \gamma S, \gamma SS, \gamma SR_{ij}, T_{cur}]$

3) $r_i \leftarrow v_{cur}^i - \frac{S+v_{cur}^i}{\omega+1}$

4) $A_\lambda^i \leftarrow [\omega+1, S+v_{cur}^i, SS+(v_{cur}^i)^2, SR_{ij}+r_i r_j, T_{cur}]$

5) return A_λ^i

1-dimensional statistics of data sequence D_i can be calculated through A_λ^i , and 2-dimensional statistics of D_i and D_j can be calculated through A_λ^i and A_λ^j . The 1-dimensional statistics include weight (ω), mean (μ_{D_i}) and variance ($\sigma_{D_i}^2$). ω is given in A_λ^i , and others are as follows:

$$\mu_{D_i} = \frac{S}{\omega} \quad (4)$$

$$\sigma_{D_i}^2 = \left| \frac{SS}{\omega} - (\mu_{D_i})^2 \right| \quad (5)$$

The 2-dimensional statistics include magnitude ($\|D_i, D_j\|$), radius (R_{D_i, D_j}), approximate covariance (Cov_{D_i, D_j}) and approximate correlation coefficient (P_{D_i, D_j}):

$$\|D_i, D_j\| = \sqrt{(\mu_{D_i})^2 + (\mu_{D_j})^2} \quad (6)$$

$$R_{D_i, D_j} = \sqrt{\sigma_{D_i}^2 + \sigma_{D_j}^2} \quad (7)$$

$$Cov_{D_i, D_j} = \frac{SR_{ij}}{\omega_i + \omega_j} \quad (8)$$

$$P_{D_i, D_j} = \frac{Cov_{D_i, D_j}}{\sqrt{\sigma_{D_i}^2} \sqrt{\sigma_{D_j}^2}} \quad (9)$$

In a given time window, the feature extractor checks four meta information for each captured packet, including source IP (SrcIP), source MAC-IP (SrcMAC-IP), source-destination IP (Channel) and source-destination TCP/UDP Socket (Socket). The packets with the same meta information are divided into one group, and then four data are extracted of the same meta information group, including the size of outbound packets (OutPktSize), the size of inbound

and outbound packets (IOPktSize), the count of packets (PktCount) and the interval time of arrived packets (PktJitter), and these data are v_j^i in D_i . We finally calculate the statistics as shown in Table 1 and get the 32-dimensional feature.

Table 1: Extraction of Basic Features

Meta Information	Data	Statistics	Number of Features
SrcIP	OutPktSize	μ, σ^2	8
SrcMAC-IP			
Channel	PktCount	ω	4
Socket			
Channel	IOPktSize	$\ D_i, D_j\ , R_{D_i, D_j}$ $Cov_{D_i, D_j}, P_{D_i, D_j}$	8
Socket			
Channel	PktJitter	μ, σ^2, ω	3

Since the features of data usually have different dimensions, the Z-Score algorithm is employed in this paper to normalize the features and eliminate the dimensional effect. In the time window corresponding to a given decay factor, the sample vector got by the feature extractor is recorded as $S_v = [s_1, s_2, \dots, s_n]^T$, and the i -th sample is:

$$s_i = [f_1^i, f_2^i, \dots, f_m^i] \quad (10)$$

where f_j^i is the j -th dimensional feature of the sample, and the sample vector can be represented as a matrix:

$$S_v = \begin{bmatrix} f_1^1 & f_2^1 & \dots & f_m^1 \\ f_1^2 & f_2^2 & \dots & f_m^2 \\ \dots & \dots & \dots & \dots \\ f_1^n & f_2^n & \dots & f_m^n \end{bmatrix} \quad (11)$$

and the formulas of Z-Score are:

$$\mu_{f_j} = \frac{1}{n} \sum_{i=1}^n f_j^i \quad (12)$$

$$\sigma_{f_j} = \sqrt{\frac{1}{n} \sum_{i=1}^n (f_j^i - \mu_{f_j})^2} \quad (13)$$

$$\tilde{f}_j^i = \frac{f_j^i - \mu_{f_j}}{\sigma_{f_j}} \quad (14)$$

3 TAM-based MCA Process

There are implicit correlations between the original features of network traffic, and the occurrence of abnormal network behaviors will change these correlations. Therefore, the multivariate correlation of original features can be treated as an important information, which can be transformed into a feature set with stronger descriptive ability [5].

TAM-based MCA is an algorithm that maps features of high-dimensional space to low-dimensional subspace, and applies the concept of triangular area to extract geometric correlations between features [6]. For the sample matrix S_v as shown in formula (11), the i -th sample s_i represents a point P_{s_i} in the m -dimensional space. We project P_{s_i} to a 2-dimensional Euclidean subspace $\mathbb{S}_{k,l}$ of the k - l axis:

$$y_{k,l}^i = \begin{bmatrix} \epsilon_k \\ \epsilon_l \end{bmatrix} s_i^T = \begin{bmatrix} f_k^i \\ f_l^i \end{bmatrix} \quad (15)$$

where $1 \leq k, l \leq m$ and $k \neq l$, ϵ_k has the following forms (and so does ϵ_l):

$$\epsilon_k = [e_{k,1}, e_{k,2}, \dots, e_{k,m}] \quad (16)$$

$$e_{k,q} = \begin{cases} 1, & k = q \\ 0, & k \neq q \end{cases} \quad (17)$$

The corresponding point for $y_{k,l}^i$ is $p_{k,l}^i(f_k^i, f_l^i)$, and the area of the triangular region $\triangle O f_k^i f_l^i$ formed by the projection point $p_{k,l}^i(f_k^i, f_l^i)$ on the k - l axis and the origin point O is defined as:

$$\begin{aligned} T_{k,l}^i &= S(\triangle O f_k^i f_l^i) \\ &= \frac{(\|(f_k^i, 0) - (0, 0)\| \times \|(0, f_l^i) - (0, 0)\|)}{2} \end{aligned} \quad (18)$$

Then $T_{k,l}^i$ can be regarded as the correlation value of f_k^i and f_l^i which are the k -th and the l -th feature of sample s_i . We finally get the multivariate correlation matrix TAM^i (see formula (19)) of s_i , which is a m -dimensional symmetric square matrix. The diagonal elements of TAM^i are all zero because we are not interested in the correlation between the feature and itself.

$$TAM^i = \begin{bmatrix} 0 & T_{1,2}^i & T_{1,3}^i & \dots & T_{1,m}^i \\ T_{2,1}^i & 0 & T_{2,3}^i & \dots & T_{2,m}^i \\ T_{3,1}^i & T_{3,2}^i & 0 & \dots & T_{3,m}^i \\ \dots & \dots & \dots & \dots & \dots \\ T_{m,1}^i & T_{m,2}^i & T_{m,3}^i & \dots & 0 \end{bmatrix} \quad (19)$$

For the network stream S_v with the sample number of n , the final form of the TAM-based MCA is:

$$S_v^{MCA} = [TAM^1, TAM^2, \dots, TAM^n]^T \quad (20)$$

We treat TAM^i as a 2-dimensional image matrix with each element representing a pixel, and use the Min-Max method to scale elements of the matrix to the interval of $[0, 255]$. Therefore, each TAM^i can be converted into a 23×23 grayscale image. In order to verify the algorithm, we make an IoT device named Danmini Doorbell work in benign mode, then we infect the device with a Mirai botnet program, and make the device execute five kinds of attacks, namely ACK flood, SCAN flood, SYN flood, UDP flood and UDPPLAIN flood. We capture the inbound and outbound traffic during the six run phases, and process the traffic by the methods referred in section 2 and 3 of this paper. We finally get the grayscale images of the traffic (see Fig. 1). It can be seen that there are significant differences between the images of benign traffic and attack traffic, and the TAM-based MCA algorithm has good performance of feature description.

4 Structure of the CNN

The features of the processed network traffic by the TAM-based MCA algorithm can be regarded as grayscale images, and the convolutional neural network in deep learning has been proved to have good performance in image recognition and classification [7][8][9]. So we design a CNN (see Fig. 2) to detect the network traffic in this paper.

Convolutional layer 1: This layer convolves the 23×23 inputs by sixteen 4×4 convolution kernels with a step size

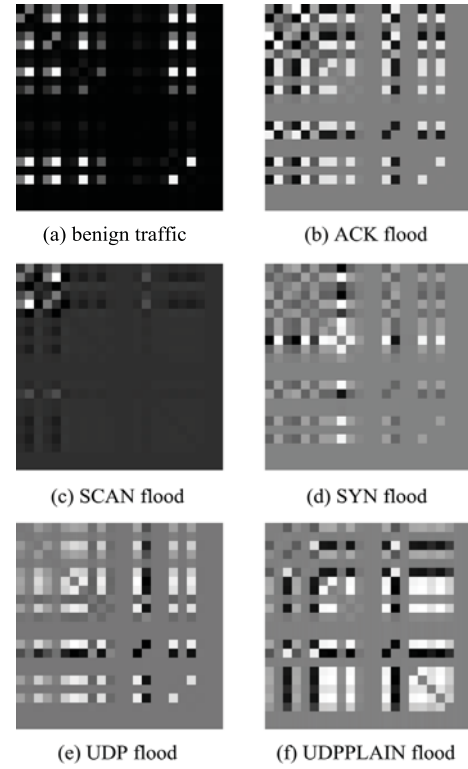


Fig. 1: The grayscale images of traffic

of 1 and uses the ReLU activation function and gets sixteen 20×20 feature maps. The function of the convolutional layer is to extract different features of the input. The higher the level of the convolutional layer, the more complex the extracted features, and the characteristics of local perception and weight-sharing can reduce the complexity of the network. The convolutional process of the j -th feature map x_j^l in the l -th layer is:

$$x_j^l = f \left(\sum_{i \in M_j} x_i^{l-1} * k_{ij}^l + b_j^l \right) \quad (21)$$

where M_j is the set of feature maps in the $(l-1)$ -th layer, and $*$ is the 2-dimensional convolution operation, and k_{ij}^l is the convolution kernel of the l -th layer, and b_j^l is the bias of the convolution kernel, and $f(\bullet)$ is the ReLU activation function.

Pooling layer 1: In this layer, a 2×2 kernel with step size of 2 was applied for downsampling of feature map. The sampling method is maximum sampling, and the size of the pooled feature map was reduced by half to 10×10 . The main purpose of pooling is to reduce the number of parameters and prevent network overfitting to some extent.

Convolutional layer 2: This layer convolves the feature maps of previous layer by using thirty-two 3×3 convolution kernels with a step size of 1 and gets thirty-two 8×8 feature maps and uses the ReLU activation function.

Pooling layer 2: Do the same work with pooling layer 1 and the size of pooled feature maps is 4×4 .

Dropout layer: Since the number of the dataset in this paper is about 480,000, in order to prevent overfitting [10], neurons are randomly interrupted from the network at this layer with the probability of $P = 0.5$. The output of the i -th neuron in the $(l+1)$ -th layer after adding the dropout is

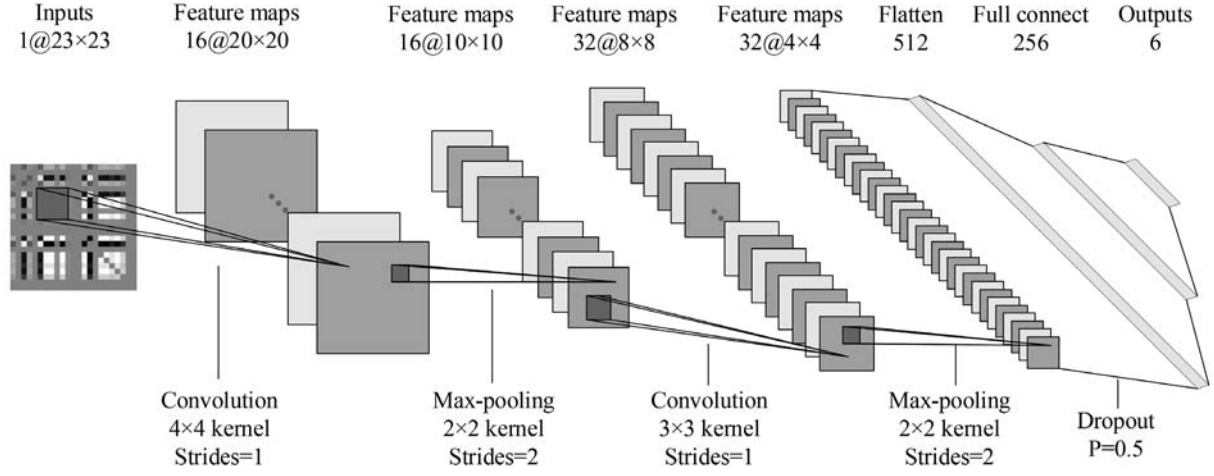


Fig. 2: The structure of the CNN

shown in formula (22)~(24):

$$r_j^l \sim \text{Bernoulli}(p) \quad (22)$$

$$\tilde{\mathbf{y}}^l = \mathbf{r}^l * \mathbf{y}^l \quad (23)$$

$$y_i^{l+1} = f(\omega_i^{l+1} \tilde{\mathbf{y}}^l + b_i^{l+1}) \quad (24)$$

where r_j^l is the possibility of the j -th input in the $(l+1)$ -th layer with the value of 0 or 1, and follow the Bernoulli distribution with a parameter of P . $\tilde{\mathbf{y}}^l$ is the actual input of the $(l+1)$ -th layer. ω_i^{l+1} and b_i^{l+1} are the weight and bias of the i -th neuron in the $(l+1)$ -th layer. y_i^{l+1} is the output of the $(l+1)$ -th layer. $f(\bullet)$ is the activation function. In addition, the dropout layer needs to be discarded during testing, and the output shall be rescaled by multiplied of P :

$$y_{train} = W|_p * x \quad (25)$$

$$y_{test} = W * Px \quad (26)$$

Flatten layer: This layer transforms thirty-two 4×4 feature maps of the previous layer into a 1×512 dimensional vector for the transition from the convolutional layer to the full connected layer.

Full connected layer: All two hundred and fifty-six neurons in this layer is fully connected to the output of the flatten layer with the ReLU activation function. The full connected layer can integrate local information in the output of previous layers.

Output layer: Softmax classifier is applied in this layer which is defined as:

$$p_j(z) = \frac{e^{z_j}}{\sum_k e^{z_k}} \quad (27)$$

where z_j is the output of the j -th neuron in previous layer and means the sample's score for all categories. $p_j(z)$ is the probability distribution of this sample for all categories. In this paper, the output have six categories, respectively, benign traffic and five attack traffic. The categorical cross-entropy is used as loss function (see formula (28)), where n means the number of categories, y_n and $\hat{y}_n$ is actual value and predicted value respectively.

$$L = -\frac{1}{N} \sum_{n=1}^N [y_n \log \hat{y}_n + (1-y_n) \log (1-\hat{y}_n)] \quad (28)$$

5 Experiments and Result Analysis

In this paper, we use the Keras deep learning framework with the backend of TensorFlow to build and train the network, and the computer was configured with a 2.3Ghz Intel i5-6300HQ CPU, an 8GB memory and a NVIDIA GeForce GTX-960M GPU with 2GB video memory.

The dataset is the IoT botnet attack dataset published on UCI Machine Learning Repository by the research team of Ben-Gurion University of the Negev [11]. The description of the dataset is shown in Table 2.

Table 2: Dataset Description

Dataset Type	Label(Traffic Type)	Quantity	Total
Train	0 (BENIGN)	28445	484746
	1 (ACK)	71355	
	2 (SCAN)	75398	
	3 (SYN)	85790	
	4 (UDP)	166253	
	5 (UDPPPLAIN)	57511	
Test	0 (BENIGN)	11952	207749
	1 (ACK)	30842	
	2 (SCAN)	32289	
	3 (SYN)	36785	
	4 (UDP)	71414	
	5 (UDPPPLAIN)	24473	

We apply the Adadelta [12] optimization algorithm to update parameters of the network with the decay factor γ of 0.95 and the ϵ of 1×10^{-7} . In addition, the batch size is 128, and the epoch is 10. The evaluation function is categorical accuracy, defined as the mean accuracy rate across all predictions for multiclass classification problems. The curves of accuracy and loss on each epoch during training and testing as shown in Fig. 3.

The final result shows that the accuracy is 98.62%, the loss is 4.74%, and the average training time is 32s per epoch on the training set, and the accuracy reaches 99.57% on the test set, while loss is 1.74% and the test time is 10s per epoch. In order to evaluate the detection performance more comprehensively, we calculate three indicators for each category,

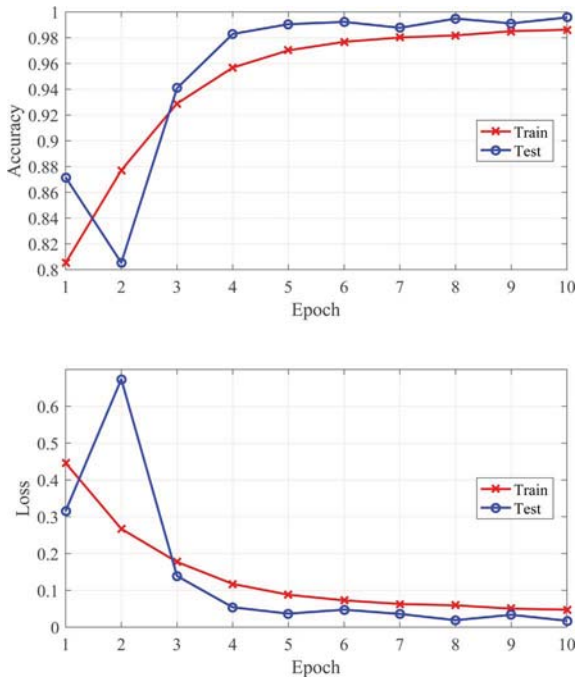


Fig. 3: The accuracy and loss curves for train and test

namely F1-Score, precision and recall (see Fig. 4). It can be seen that for each type of traffic, the method in this paper has achieved a good performance. Although the score of ACK flood and UDP flood are significantly lower than others, we guess that there are more similar frames between the two types of attack traffic after preprocessing, and the lowest score is still above 0.95. Most abnormal detection systems only perform binary classification of benign traffic and attack traffic. In this paper, multiple classification is implemented to determine the attack type and enhance the functionality of the system.

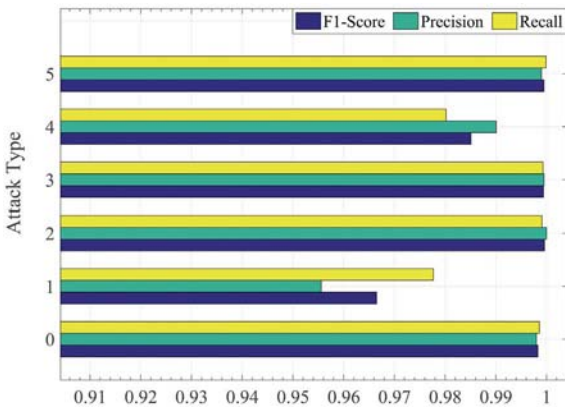


Fig. 4: F1-Score, precision and recall for each category

We also compare our method with other works. In the first contrast experiment, we use the same dataset to train a three-layer MLP with one input layer (flatten the 23×23 input matrix into a 1×529 vector), two hidden layers with 16 and 32 neurons respectively and one output layer. In the second contrast experiment, we use the method referred in [3] that apply the Euclidean distance (ED) instead of TAM to calculate the features' multivariate correlation coefficient and utilize CNN with the same structure referred in our pa-

per. The results are shown in Table 3, it can be seen that our method has a better performance.

Table 3: The accuracy of three methods

Our method	CNN with ED-based MCA	MLP
99.57%	97.76%	96.13%

6 Conclusions

We use the damped incremental statistics and the Z-Score method to extract and normalize the 23-dimensional basic features of inbound and outbound traffic (including benign traffic and five kinds of attack traffic) of IoT devices. Then we apply the TAM-based MCA algorithm to correlate the original 1×23 dimensional features and generate 23×23 dimensional feature matrixes, and each matrix can be regarded as a grayscale image, and the grayscale images generated from different types of traffic have obvious difference. Based on this premise, we design a convolutional neural network to learn the MCA matrixes, and the final experiments achieve satisfactory result. Therefore, the approach with preprocessing and CNN proposed in this paper have a good performance in IoT botnet detection.

References

- [1] C. Livadas, R. Walsh, D. Lapsley, et al, Using machine learning techniques to identify botnet traffic, *Local Computer Networks, Proceedings 2006 31st IEEE Conference on*, 2006: 967-974.
- [2] C. Zhou, Z. Huang, Deep learning detection based on traffic characteristics of botnet, *Information Technology*, 2018(4): 1-9.
- [3] G. Kou, G.M. Tang, S. Wang, et al, Using deep learning for detecting BotCloud, *Journal on Communications*, 37(11): 114-128, 2016.
- [4] Y. Mirsky, T. Doitshman, Y. Elovici, et al, Kitsune: An ensemble of autoencoders for online network intrusion detection, *Network and Distributed System Security (NDSS) Symposium*, 2018.
- [5] Z. Tan, A. Jamdagni, X. He, et al, Detection of denial-of-service attacks based on computer vision techniques, *IEEE Transactions on Computers*, 64(9): 2519-2533, 2015.
- [6] Z. Tan, A. Jamdagni, X. He, et al, A system for denial-of-service attack detection based on multivariate correlation analysis, *IEEE transactions on parallel and distributed systems*, 25(2): 447-456, 2014.
- [7] Y. Lecun, Y. Bengio, Convolutional networks for images, speech, and time series, *The Handbook of Brain Theory and Neural Networks*, 3361(10), 1995.
- [8] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Advances in Neural Information Processing Systems*, 2012: 1097-1105.
- [9] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, *International Conference for Learning Representations (ICLR)*, 2015.
- [10] N. Srivastava, G.E. Hinton, A. Krizhevsky, et al, Dropout: A simple way to prevent neural networks from overfitting, *The Journal of Machine Learning Research*, 15(1): 1929-1958, 2014.
- [11] Y. Meidan, M. Bohadana, Y. Mathov, et al, N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders, *IEEE Pervasive Computing*, 17(3): 12-22, 2018.
- [12] M.D. Zeiler, ADADELTA: an adaptive learning rate method, *Computing Research Repository (CoRR)*, arXiv: 1212.5701, 2012.